

CLAUDE CO-WORK SUB-AGENTS MASTERCLASS

Build Your Own AI Agent Army Without Writing Code

Run parallel AI agents, create custom skills, and persist your workflows - all from the Claude desktop app.

By Mark Kashef

Early AI Adopters Community

TABLE OF CONTENTS

1. Introduction: Why Sub-Agents Matter	3
2. Understanding Claude Co-work's Agent Architecture	4
3. Built-In Sub-Agents Reference	6
4. Creating Custom Sub-Agents	9
5. The Agent Army Workflow Pattern	12
6. Real-World Use Cases & Examples	15
7. Persisting Agents for Reuse	20
8. Context Window Management	23
9. Advanced Techniques & Best Practices	25
10. Troubleshooting Common Issues	28
Appendix: Quick Reference Cheat Sheet	30

1. INTRODUCTION: WHY SUB-AGENTS MATTER

If you've been using Claude through the desktop app, you've likely experienced a common frustration: complex tasks that require multiple steps can quickly eat through your context window, forcing auto-compaction and losing valuable conversation history.

Sub-agents solve this problem elegantly. Instead of executing everything in a single conversation thread, you can spawn multiple specialized agents that work in parallel, each with their own fresh context window. This is the same powerful capability that Claude Code users have been leveraging - but now accessible without touching a terminal.

What You'll Learn

- How to spawn multiple AI agents that work simultaneously on different tasks
- The built-in sub-agents available in Claude Co-work and when to use each
- How to create custom agents tailored to your specific workflows
- Techniques for persisting agents so you can reuse them across sessions
- Real-world examples including research, content creation, and analysis
- Best practices for orchestrating complex multi-agent workflows

PRO TIP: This guide assumes you have access to Claude Co-work (the desktop app's agentic mode). If you're still using the standard chat interface, you'll need to enable Co-work mode first.

2. UNDERSTANDING CLAUDE CO-WORK'S AGENT ARCHITECTURE

Before diving into practical applications, it's essential to understand how Claude Co-work's agent system is structured. This mental model will help you design more effective workflows.

The Orchestrator-Worker Model

When you interact with Claude Co-work, you're communicating with an orchestrator agent. This primary agent has the ability to:

- **Spawn sub-agents** - Create specialized workers for specific tasks
- **Delegate work** - Assign tasks to sub-agents based on their capabilities
- **Coordinate results** - Gather outputs from multiple agents and synthesize them
- **Manage context** - Keep the main conversation clean while agents work independently

Sequential vs. Parallel Execution

Sequential Execution (Traditional Approach): Each step must complete before the next begins. Research A finishes, then Research B starts, then synthesis happens. This is slow and context-heavy.

Parallel Execution (Sub-Agent Approach): Multiple research agents work simultaneously. Research A, B, and C all run at the same time. Results are gathered and synthesized only after all agents complete. This is faster and preserves context.

Aspect	Sequential	Parallel (Sub-Agents)
Speed	Slow (tasks wait)	Fast (simultaneous)
Context Usage	High (everything in one thread)	Low (distributed)
Visibility	All in main chat	Expandable sub-tasks
Error Impact	Blocks entire flow	Isolated to agent
Complexity	Simple prompts	Requires orchestration

How Sub-Agents Appear in the UI

When sub-agents are spawned, they appear as expandable cards in the Claude Co-work interface. Each card shows:

- The agent's assigned task/name
- Current status (working, completed, or waiting)
- Expandable steps showing exactly what the agent is doing
- Sources and references the agent consulted
- Final output/results when complete

PRO TIP: You can click on any sub-agent card to expand it and see detailed step-by-step progress. This is invaluable for debugging or understanding how Claude approaches different problems.

3. BUILT-IN SUB-AGENTS REFERENCE

Claude Co-work comes with several pre-configured sub-agents out of the box. Understanding their strengths helps you choose the right tool for each task.

General Purpose Research Agent

The workhorse of the sub-agent system. This agent excels at web research, information synthesis, and multi-step exploration. It can spawn its own sub-agents when needed, creating a cascading research structure.

Best For: Deep research tasks, competitive analysis, market research, fact-finding missions, exploring unfamiliar topics

Example Prompt: *"Research the top 5 AI automation trends for 2026 and summarize the key findings from credible sources."*

Plan Agent

A strategic thinking agent designed to structure complex problems into actionable steps. It analyzes requirements, identifies dependencies, and creates logical execution sequences.

Best For: Project planning, roadmap creation, breaking down complex tasks, strategic analysis, workflow design

Example Prompt: *"Create a 90-day AI adoption roadmap for a mid-size marketing agency with 50 employees."*

Explore Agent

Specialized for codebase exploration and technical investigation. While primarily designed for developers, it's also useful for understanding complex systems and documentation structures.

Best For: Code analysis, documentation review, technical exploration, understanding system architecture

Example Prompt: *"Explore this codebase and explain how the authentication system works."*

Bash Agent

Executes terminal commands and system operations. Useful when you need to interact with your local file system, run scripts, or perform system-level tasks.

Best For: File operations, running scripts, system administration, creating folder structures

Example Prompt: *"Create a new project folder structure with subdirectories for research, outputs, and assets."*

Built-In Skills (Not Sub-Agents)

In addition to sub-agents, Claude Co-work includes built-in skills for document creation. These aren't agents but capabilities that can be invoked:

Skill	File Type	Use Case
PowerPoint (PPTX)	.pptx	Presentations, slide decks, pitch decks
Word (DOCX)	.docx	Reports, proposals, documentation
Excel (XLSX)	.xlsx	Data analysis, spreadsheets, calculations
PDF	.pdf	Final documents, forms, printable content

IMPORTANT: Skills and sub-agents are different! Skills are capabilities (like creating a PowerPoint), while sub-agents are workers (like a researcher). You can have a sub-agent use a skill as part of its work.

4. CREATING CUSTOM SUB-AGENTS

The real power of Claude Co-work emerges when you create custom sub-agents tailored to your specific needs. Here's how to design and deploy your own agent army.

The Agent Definition Framework

Every effective custom agent needs four components:

- **Name** - A clear, descriptive identifier (e.g., 'Content Reviewer Agent')
- **Role** - What the agent is responsible for doing
- **Capabilities** - Specific tasks the agent can perform
- **Constraints** - Boundaries and quality standards

Example: Creating a Content Reviewer Agent

Here's a prompt that creates a QA-focused sub-agent:

```
I want you to create a Content Reviewer Agent with the following specifications:
```

```
NAME: Content Reviewer Agent
```

```
ROLE: Review created documents (PowerPoints, Word docs, etc.) for quality issues before final delivery.
```

```
CAPABILITIES:
```

- Detect text overlaps and overflow issues
- Identify poor color contrast (e.g., white text on light backgrounds)
- Flag inconsistent formatting across slides/pages
- Check for spelling and grammar errors
- Verify all content is within visible boundaries

```
CONSTRAINTS:
```

- Must provide specific locations for each issue found
- Should categorize issues by severity (Critical, Warning, Minor)
- Must not modify the original document, only report findings

Common Custom Agent Types

- **Research Specialist:** Focus on a specific domain (AI trends, competitor analysis, market research)

- **Quality Assurance:** Review outputs for errors, inconsistencies, or issues
- **Content Transformer:** Convert content between formats or styles
- **Data Analyst:** Process and analyze structured data
- **Summarizer:** Condense long content into key points
- **Fact Checker:** Verify claims and sources

5. THE AGENT ARMY WORKFLOW PATTERN

The most powerful way to use sub-agents is through orchestrated workflows. Here's the pattern that maximizes efficiency and output quality.

The Four-Phase Pattern

Phase	Agents Used	Output
1. RESEARCH	Multiple General Purpose agents in parallel	Raw findings from different angles
2. PLAN	Plan Agent	Structured roadmap/outline
3. EXECUTE	Skills (PPTX, DOCX, etc.)	Draft deliverable
4. QA	Content Reviewer Agent	Final polished output

Example: Executive Presentation Workflow

Let's walk through creating an AI adoption roadmap presentation for a C-suite audience:

Phase 1: Parallel Research

Spawn 3-4 research agents simultaneously:

- Agent 1: Research emerging AI capabilities expected in 2026
- Agent 2: Research AI adoption challenges and failure patterns
- Agent 3: Research successful enterprise AI implementation case studies
- Agent 4: Research industry-specific AI opportunities

Phase 2: Strategic Planning

The Plan Agent synthesizes research findings into a structured roadmap with quarterly milestones, resource requirements, and success metrics.

Phase 3: Content Creation

The PPTX skill transforms the plan into a polished executive presentation with consistent branding and clear visualizations.

Phase 4: Quality Assurance

The Content Reviewer Agent checks for visual issues, text overlaps, color contrast problems, and content consistency.

PRO TIP: Always include a QA phase! Even one-shot generations can have subtle issues like text overflow or poor color contrast that are easy to miss but unprofessional to present.

6. REAL-WORLD USE CASES & EXAMPLES

Here are detailed examples of how to apply sub-agents to common business and creative tasks.

Use Case 1: Competitive Analysis Report

Scenario: You need to analyze 5 competitors and create a comprehensive report.

Traditional Approach: Research each competitor sequentially, taking 5x longer.

Sub-Agent Approach:

I need to conduct a competitive analysis of these 5 companies:
[Company A, B, C, D, E].

Please spawn 5 parallel research agents, one for each company. Each agent should investigate:

- Product/service offerings
- Pricing strategy
- Target market
- Key differentiators
- Recent news/developments
- Strengths and weaknesses

Once all research is complete, synthesize the findings into a comparison matrix and strategic recommendations document using the DOCX skill.

Use Case 2: Content Repurposing Pipeline

Scenario: Transform a long-form blog post into multiple content formats.

I have a 3000-word blog post about [topic]. I need to repurpose it into multiple formats.

Please create these parallel agents:

1. Social Media Agent: Extract 10 tweet-worthy quotes/insights
2. LinkedIn Agent: Create a thought leadership post version
3. Email Agent: Write a newsletter summary with key takeaways
4. Slide Agent: Create a 10-slide presentation of main points

Run all agents simultaneously, then compile outputs into a single "Content Package" document.

Use Case 3: Research Synthesis for Decision Making

Scenario: Evaluate whether to adopt a new technology for your business.

I'm considering adopting [Technology X] for my business. Help me make an informed decision.

Spawn these research agents in parallel:

1. Benefits Research Agent: Find documented benefits and success stories
2. Risk Research Agent: Identify potential risks, failures, and challenges
3. Cost Research Agent: Research pricing, implementation costs, and ROI data
4. Alternative Research Agent: Find competing solutions and compare features

After research completes, use the Plan Agent to create a decision framework, then generate an executive summary using the DOCX skill.

Use Case 4: Event/Workshop Planning

I'm planning a 2-day AI workshop for 50 executives. Help me plan everything.

Create these parallel planning agents:

1. Content Agent: Research and outline workshop modules and activities
2. Logistics Agent: Create checklist for venue, catering, A/V needs
3. Materials Agent: Design participant workbooks and handouts
4. Follow-up Agent: Plan post-workshop engagement and resources

Synthesize into a comprehensive event planning document with timelines.

Use Case 5: Client Proposal Generation

I need to create a proposal for [Client Name] for [Project Type].

Spawn these agents:

1. Discovery Agent: Research the client's company, challenges, and industry
2. Solution Agent: Design a tailored approach based on their needs
3. Pricing Agent: Create a pricing structure with options
4. Case Study Agent: Find relevant success stories to include

Once complete, create a professional proposal using the DOCX skill, then generate an accompanying slide deck with the PPTX skill for the presentation meeting.

PRO TIP: For client-facing work, always add a QA agent at the end to catch any errors before delivery. One typo can undermine an otherwise excellent proposal.

7. PERSISTING AGENTS FOR REUSE

One of the most powerful but underutilized features is the ability to save your custom agents and workflows for reuse across sessions. Here's how to create a persistent agent library.

Setting Up Your Agent Library

Step 1: Create a dedicated folder for your Co-work agents:

```
cowork-agents/  
■■■ agents.md # Agent definitions  
■■■ skills/ # Custom skills  
■■■ workflows/ # Saved workflow patterns  
■■■ README.md # Quick reference guide
```

Step 2: After a successful session, ask Claude to document everything:

```
Please document everything we created in this session:  
  
1. Create an agents.md file with detailed descriptions of all  
sub-agents we used, including:  
- Agent name and role  
- When to use this agent  
- Example prompts to invoke it  
- Any special instructions or constraints  
  
2. Save any custom skills to the skills/ folder  
  
3. Create a README.md that a new Co-work session can read to  
understand how to use these agents  
  
Store everything in our cowork-agents folder.
```

Step 3: In future sessions, onboard quickly:

```
Please read the README.md and agents.md files in my cowork-agents  
folder and onboard yourself on how to use each agent and skill listed  
there.
```

Example agents.md Structure


```
# My Custom Agents Library
```

```
## Research Agents
```

```
### Market Research Agent
```

```
**Role:** Deep-dive research on market trends and opportunities
```

```
**Best For:** Market sizing, trend analysis, opportunity  
identification
```

```
**Invoke With:** "Spawn a market research agent to investigate  
[topic]"
```

```
### Competitor Analysis Agent
```

```
**Role:** Investigate competitor strategies and positioning
```

```
**Best For:** Competitive intelligence, feature comparisons
```

```
**Invoke With:** "Create a competitor analysis agent for [company]"
```

```
## QA Agents
```

```
### Content Reviewer Agent
```

```
**Role:** Review documents for quality issues
```

```
**Capabilities:** Text overlap detection, color contrast, formatting
```

```
**Invoke With:** "Run the content reviewer agent on [document]"
```

IMPORTANT: Keep your agent definitions focused! If you make them too specific to one project, they won't be reusable. Aim for 'evergreen' definitions that work across similar use cases.

8. CONTEXT WINDOW MANAGEMENT

One of the biggest advantages of sub-agents is how they help manage Claude's context window. Understanding this can dramatically improve your experience.

The Context Window Problem

In a standard Claude conversation, everything accumulates in one context window. As you work through complex tasks, you'll eventually see this message:

"This conversation has been auto-compacted to save context..."

When this happens, earlier parts of your conversation are summarized or removed, potentially losing important context and making Claude 'forget' earlier decisions.

How Sub-Agents Help

Each sub-agent operates with its own fresh context window. This means:

- Research agents can dive deep without bloating your main conversation
- Multiple parallel agents don't compete for the same context space
- Your orchestrator conversation stays clean and focused
- You can run longer, more complex workflows without compaction
- Each agent's context is optimized for its specific task

Best Practices for Context Efficiency

- **Delegate heavy research to sub-agents:** Instead of asking the orchestrator to research, spawn dedicated research agents
- **Keep the orchestrator for coordination:** Use your main chat for high-level direction, not detailed work
- **Use sub-agents for iterative tasks:** Multiple rounds of revision? Use an agent, not the main thread
- **Clear completed sub-agent context:** Sub-agent contexts are cleared when they finish, automatically freeing space

9. ADVANCED TECHNIQUES & BEST PRACTICES

Technique 1: Cascading Agent Structures

Sub-agents can spawn their own sub-agents. This creates powerful cascading structures for complex research:

```
Create a master research agent that will:  
1. First identify the 5 most important subtopics to investigate  
2. Then spawn a dedicated sub-agent for each subtopic  
3. Finally synthesize all findings into a comprehensive report  
  
The master agent should coordinate the sub-agents and ensure no  
duplicate effort.
```

Technique 2: Agent Specialization Chains

Chain specialized agents together where each one's output feeds the next:

1. Research Agent produces raw findings
2. Analysis Agent identifies patterns and insights
3. Strategy Agent develops recommendations
4. Content Agent creates deliverables
5. QA Agent validates quality

Technique 3: Mid-Workflow Intervention

Use the chat intervention feature to adjust course without restarting:

While agents are running, click the intervention button to add context, correct direction, or expand scope. This is especially useful when you see agents going in an unexpected direction.

Technique 4: Template-Based Workflows

Create reusable workflow templates that you can quickly customize:

WORKFLOW TEMPLATE: Client Deliverable

Variables to customize:

- [CLIENT_NAME]
- [PROJECT_TYPE]
- [DELIVERABLE_FORMAT]
- [DEADLINE]

Agents to spawn:

1. Research Agent: Investigate [CLIENT_NAME]'s industry and challenges
2. Solution Agent: Design [PROJECT_TYPE] approach
3. Content Agent: Create [DELIVERABLE_FORMAT]
4. QA Agent: Review and polish

Execute workflow with deadline awareness for [DEADLINE].

10. TROUBLESHOOTING COMMON ISSUES

Issue	Cause	Solution
Agents not spawning	Unclear instruction	Explicitly say "spawn a sub-agent for..."
Sequential instead of parallel	Ambiguous phrasing	Say "spawn these agents simultaneously" or "in parallel"
Agents duplicating work	Overlapping scope	Define clear, distinct responsibilities for each agent
QA missing issues	Vague QA criteria	Specify exact checks: "look for text overflow, color contrast..."
Context compaction still occurs	Too much in main thread	Delegate more work to sub-agents
Inconsistent outputs	No style guide	Provide examples or templates in your prompts

Quick Diagnostic Prompts

If things aren't working as expected, try these:

- "Can you confirm what sub-agents are currently available?"
- "Show me the status of all running agents"
- "Why did you choose sequential execution instead of parallel?"
- "What specific tasks is each agent working on right now?"

APPENDIX: QUICK REFERENCE CHEAT SHEET

Magic Phrases That Trigger Sub-Agents

- "Spawn [N] agents in parallel to..."
- "Create a sub-agent that specializes in..."
- "Run these agents simultaneously..."
- "Create a [Name] Agent with the role of..."
- "After all agents complete, synthesize..."

The 4-Phase Workflow (Memorize This)

Phase	Action	Agent Type
1. RESEARCH	Gather information	General Purpose (parallel)
2. PLAN	Structure approach	Plan Agent
3. EXECUTE	Create deliverable	Skills (PPTX/DOCX/etc)
4. QA	Review & polish	Content Reviewer

Persistence Commands

- "Document all agents we used into agents.md"
- "Create a README for future sessions"
- "Save this workflow as a reusable template"
- "Read the README and onboard yourself on my agents"

Want more AI automation tutorials and early access to new techniques?

Join the Early AI Adopters Community:

<https://www.skool.com/earlyadopters/about>